

Inferencelens

Interview Defense Pack · v2

Inference Cost / Quality Tradeoff Auditor

Makes the Pareto frontier explicit, measurable, and defensible

Thesis question	How does an AI system know it's working correctly when cost and quality compete?
Core output	Pareto frontier + IF/THEN routing rules + append-only JSONL audit trail
Pareto dominance	Config A dominates B: $\text{cost}(A) < \text{cost}(B)$ AND $\text{quality}(A) \geq \text{quality}(B)$
Model tiers	large (gpt-4o) · medium (gpt-4o-mini) · small (gpt-3.5-turbo) · local (ollama/mistral)
Complexity score	$0.50 \cdot \text{token_score} + 0.30 \cdot \text{vocab_entropy} + 0.20 \cdot \text{task_baseline}$; range 0–1
Routing bands	Simple $<0.35 \rightarrow$ local/small · Standard $0.35\text{--}0.70 \rightarrow$ medium · Complex $\geq 0.70 \rightarrow$ large
ROUGE-1	Summarization quality: unigram overlap F1; pure Python; no transformer deps
Dataset	300 prompts · 3 task types · 4 model configs · seed=42
Quality floors	Summarization: large=0.91/medium=0.88/small=0.79/local=0.74
Audit verdict	PASS (optimal frontier) · WARN (quality drop $>10\%$ or savings $<20\%$) · FAIL (dominated)
Savings floor	20% minimum savings to route down a tier (config.py ROUTING_THRESHOLDS)
Tests	48 passing — pipeline integration, Pareto logic, routing, evaluator

Sidharth Kriplani · May 2026 · github.com/sidharthkriplani/inferencelens

Section 1 — The Problem: AI Routing Without Auditability

Every production AI system makes implicit cost/quality tradeoffs on every inference call. Most engineering teams route all calls to the largest available model by default, or apply hand-tuned heuristics with no systematic validation. Neither approach produces an auditable paper trail of why a routing decision was made, whether it is cost-efficient, or whether quality has silently degraded as model pricing changed.

1.1 The Core Failure Mode

When gpt-4o handles a sentiment classification that gpt-3.5-turbo would solve at 94% quality for 97% less cost — that is a routing failure. At 1 million daily calls, a 97% cost reduction on 40% of tasks represents \$40,000+/month in unnecessary spend. But without empirical profiling, nobody knows. The routing decision was made implicitly; there is no evidence trail.

The inverse failure is equally dangerous: a complex extraction task routed to a local model achieving 0.70 quality vs. 0.89 for the large model — and nobody catches it because there is no quality floor enforcement or monitoring.

1.2 Failure Mode Taxonomy

Failure Mode	What Goes Wrong	InferenceLens Fix
Over-routing to large	97% cost premium on tasks smaller model handles at near-identical quality	Pareto frontier identifies all dominated configs; quantifies exact savings opportunity
Under-routing to small	Quality below floor for high-stakes tasks (extraction, legal, medical). Silent regression.	Quality floor enforcement per complexity band. WARN verdict fires on quality delta >10%.
No audit trail	Cannot explain why a model was chosen. No paper trail for compliance or debugging.	Append-only JSONL audit per call: prompt_id, task_type, complexity_score, tier, rationale.
Silent quality regression	Model provider updates behavior; routing rules become suboptimal. No detection.	Scheduled re-profiling. Rolling quality monitoring via audit records.
Hand-tuned heuristics	'Use small model for <100 tokens' — arbitrary threshold, no empirical validation.	Complexity-based routing bands derived from Pareto analysis of real quality measurements.

1.3 The Thesis Question

InferenceLens answers: 'How does an AI system know it's working correctly when cost and quality are competing constraints?' Answer: by profiling inference across model configurations, measuring quality degradation as cost decreases, finding the Pareto frontier empirically, and generating routing recommendations with an audit trail that makes every decision traceable.

1.4 Scope: What InferenceLens Is and Is Not

IS: an analytics and decision-support library for inference cost/quality optimization. Generates routing rules backed by empirical evidence. Audits every routing decision.

IS NOT: a live API proxy, a load balancer, a model serving infrastructure, or a real-time routing engine. InferenceLens generates the rules that a proxy like OpenRouter or LiteLLM would enforce. It is the analytics layer, not the execution layer.

Section 2 — Architecture Deep Dive

2.1 Component Map

Component	File	Role
InferenceProfiler	pipeline/profiler.py	Orchestrates task execution across model configs. Logs tokens, latency, cost per call. Returns ProfileRun objects.
QualityEvaluator	pipeline/evaluator.py	Task-specific quality: ROUGE-1 F1 (summarization), macro F1 (classification), field exact match (extraction). TF-IDF cosine fallback.
CostTracker	pipeline/cost_tracker.py	USD cost: $(\text{input_tokens}/1K) * \text{input_cost} + (\text{output_tokens}/1K) * \text{output_cost}$. Aggregates by tier and task type.
ParetoAnalyzer	pipeline/pareto.py	Dominance check: $\text{cost}(A) = \text{quality}(B)$. Returns ConfigPoint list with FRONTIER/DOMINATED.
RoutingRecommender	pipeline/router.py	Maps complexity bands (LOW/MEDIUM/HIGH) to frontier tiers. <code>_select_tier_for_band()</code> per band. RoutingRule.human_readable output.
AuditLogger	pipeline/audit.py	Append-only JSONL. AuditEvent per call: event_id, timestamp, task_type, prompt_id, tier, verdict, cost_usd, quality_score.
ReportGenerator	pipeline/report.py	Assembles InferenceLensReport: Pareto curves, routing rules, PASS/WARN/FAIL verdict, text summary.
config.py	config.py	MODEL_CONFIGS (4 tiers), QUALITY_BASELINES (per task/tier), ROUTING_THRESHOLDS, EVAL_WEIGHTS, TASK_TYPES.
data/generator.py	data/generator.py	300-prompt deterministic dataset: 100 per task type. seed=42. simulated_quality per tier from QUALITY_BASELINES.

2.2 Data Flow — Detailed

- **Step 1: Dataset generation:** data/generator.py creates 300 PromptRecord objects with ground_truth and pre-computed simulated_quality scores per tier (seed=42). Quality values follow QUALITY_BASELINES from config.py, calibrated to realistic relative degradation curves.
- **Step 2: Inference profiling:** InferenceProfiler.profile_task() iterates over all (prompt, tier) combinations. Per call: compute complexity_score, call CostTracker.estimate_cost() using ModelCostConfig.estimate_cost(input_tokens, output_tokens), call QualityEvaluator.evaluate(). ProfileRun accumulates ProfiledCall objects per tier.
- **Step 3: Pareto analysis:** ParetoAnalyzer.analyze(run) computes avg_cost, avg_quality, avg_latency per tier via ProfileRun.avg_cost(tier) etc. Nested loop applies dominance criterion. $\text{routing_efficiency} = \text{quality} / (\text{cost} * 1000 + 0.001)$. Cost savings and quality delta vs. large tier computed for each point.
- **Step 4: Routing rule generation:** RoutingRecommender.generate_rules(pareto_result) generates 3 rules (one per band). `_select_tier_for_band()`: LOW → cheapest above quality floor; MEDIUM → middle quality point; HIGH → highest quality point. `_compute_verdict()` applies PASS/WARN/FAIL logic. `_format_rule()` generates human-readable IF/THEN string.
- **Step 5: Audit logging:** AuditLogger.log() appends AuditEvent as JSONL line. Each record includes: event_id (UUID), timestamp (UTC ISO), event_type, task_type, prompt_id, tier, model_id, verdict, cost_usd, latency_ms,

quality_score, complexity_score, routing_rule_id.

- **Step 6: Report assembly:** ReportGenerator.generate() builds InferenceLensReport: per-task-type Pareto curves, routing rules with verdicts, estimated_avg_savings_pct, estimated_avg_quality_delta, global PASS/WARN/FAIL verdict, and text summary with all IF/THEN rules printed.

2.3 Model Configuration Table

Tier	Model	Input cost	Output cost	Latency	Quality (s/c/e)	Notes
large	gpt-4o	\$0.005/1K in	\$0.015/1K out	1,240ms	0.91/0.94/0.89	Highest quality, highest cost. Use for complex extraction, high-stakes classification.
medium	gpt-4o-mini	\$0.00015/1K	\$0.0006/1K	380ms	0.88/0.91/0.85	Best ROI on frontier for most tasks. Saves 98.5% vs large on summarization.
small	gpt-3.5-turbo	\$0.0005/1K	\$0.0015/1K	210ms	0.79/0.86/0.77	DOMINATED by medium for summarization (higher cost, lower quality). Task-specific.
local	ollama/mistral	\$0.00	\$0.00	890ms	0.74/0.78/0.70	Frontier (zero cost) but below quality floor (0.75) for summarization.

2.4 AuditEvent Schema — What Gets Logged Per Inference Call

Every inference decision produces an AuditEvent. The full schema (from pipeline/audit.py):

Field	Type and Description
event_id	[str] UUID-based unique identifier: 'evt_{10-char hex}'
timestamp	[str] UTC ISO 8601 timestamp of event creation
event_type	[enum] INFERENCE_CALL / ROUTING_DECISION / QUALITY_EVALUATION / PARETO_ANALYSIS / REPORT_GENERATED
task_type	[str] summarization / classification / extraction
prompt_id	[str] ID from the dataset; links call to original prompt
tier	[str] large / medium / small / local
model_id	[str] specific model name (e.g., 'gpt-4o-mini')
verdict	[enum] PASS / WARN / FAIL
cost_usd	[float] computed cost for this specific call
latency_ms	[int] simulated latency in milliseconds
quality_score	[float] 0–1 composite quality score
complexity_score	[float] 0–1 pre-call complexity estimate
routing_rule_id	[str] '{task_type}_{band}' — e.g., 'summarization_low'

cost_savings_pct	[float] % savings vs. large model for this routing decision
quality_delta	[float] quality diff vs. large (negative = worse than large)
notes	[str] freeform; used for cold_start_default, exception messages

Section 3 — Algorithm Explanations with Worked Examples

3.1 Pareto Frontier — Mathematical Definition

The Pareto frontier (from Vilfredo Pareto, 1896) is the set of solutions where no objective can be improved without worsening another. In the cost/quality space, a configuration on the Pareto frontier cannot be improved on cost without sacrificing quality, and cannot be improved on quality without increasing cost.

Formal dominance criterion (from pipeline/pareto.py):

```
Config A dominates Config B iff:
A.avg_cost < B.avg_cost AND A.avg_quality >= B.avg_quality
OR
abs(A.avg_cost - B.avg_cost) < 1e-9 AND A.avg_quality > B.avg_quality

Config is FRONTIER iff no other config dominates it.
Config is DOMINATED iff at least one other config dominates it.
```

The frontier is not a single point — it is a set. For N model configs, the frontier contains those configs on the cost-quality efficient boundary. All dominated configs are removed from routing consideration.

3.2 Worked Pareto Analysis — Summarization Task

Starting from config.py quality baselines and cost table. Avg cost per call is computed from: $(\text{avg_input_tokens}/1000) \times \text{input_cost_per_1k} + (\text{avg_output_tokens}/1000) \times \text{output_cost_per_1k}$.

Tier	Avg Cost	Quality	Dominance Analysis
large	\$0.0142	0.91	Check vs medium: cost \$0.00021 < \$0.0142 AND quality 0.88 < 0.91 — medium NOT dominating (quality worse). FRONTIER.
medium	\$0.00021	0.88	Check vs small: cost \$0.00050 > \$0.00021 — small NOT cheaper. Check vs large: quality 0.91 > 0.88 but cost worse. FRONTIER.
small	\$0.00050	0.79	Medium: cost \$0.00021 < \$0.00050 (cheaper) AND quality 0.88 > 0.79 (better). DOMINATED by medium.
local	\$0.00	0.74	No config has cost < \$0 (local is free). FRONTIER — despite lowest quality.

Result: Frontier = {large, medium, local}. Dominated = {small}. Note: local is on the frontier despite lowest quality because nothing is cheaper (\$0). However, local quality (0.74) is below the quality floor (0.75) so it cannot be recommended for standard or complex routing bands.

3.3 Routing Rule Generation — Full Algorithm

RoutingRecommender._select_tier_for_band() implements the tier selection per band:

```
def _select_tier_for_band(band, frontier_points, large_point):
    # frontier_points sorted by quality DESC
    quality_floor = 0.75 # from ROUTING_THRESHOLDS

    if band == LOW:
        eligible = [p for p in frontier_points if p.avg_quality >= quality_floor]
        eligible.sort(key=lambda p: p.avg_cost_usd) # cheapest first
        return eligible[0] # cheapest above floor

    elif band == MEDIUM:
        if len(frontier_points) >= 3:
            return frontier_points[len(frontier_points) // 2] # middle
```

```

elif len(frontier_points) >= 2:
    return frontier_points[1]
else:
    return frontier_points[0]

else: # HIGH
    return frontier_points[0] # highest quality on frontier

```

Applied to summarization (frontier sorted by quality DESC: [large(0.91), medium(0.88), local(0.74)]):

- LOW band: eligible (above 0.75 floor) = [large(0.91), medium(0.88)]. Cheapest = medium (\$0.00021). → Route MEDIUM.
- MEDIUM band: 3 frontier points, middle index = 1 → frontier_points[1] = medium. → Route MEDIUM.
- HIGH band: frontier_points[0] = large (0.91). → Route LARGE.

3.4 Complexity Scorer — Three Components

The complexity score predicts how much a prompt benefits from a larger model. Computed before any model call — zero-cost, deterministic, loggable.

Component	Weight	Formula	Rationale
token_count_score	0.50	$\min(\text{prompt_tokens} / 500, 1.0)$	50-token prompt → 0.10. 500-token prompt → 1.0 (capped). Rationale: longer prompts need wider attention and context retention.
vocabulary_entropy	0.30	$\text{unique_tokens} / \text{total_tokens}$	High TTR (0.8+): technical/domain-specific language → smaller models lose precision. Low TTR (0.2–0.4): repetitive/formulaic → smaller models handle well.
task_type_baseline	0.20	extraction=0.15 / classification=0.10 / summarization=0.05	Extraction most sensitive to model quality (structured field output). Summarization most tolerant (paraphrase acceptable).

Worked example — 300-token classification prompt, TTR=0.60:

```

token_count_score = min(300/500, 1.0) = 0.60
vocabulary_entropy = 0.60
task_type_baseline = 0.10 (classification)
complexity_score = 0.50*0.60 + 0.30*0.60 + 0.20*0.10 = 0.30 + 0.18 + 0.02 = 0.50
Band = MEDIUM (0.35 <= 0.50 < 0.70) → Route to medium (gpt-4o-mini)

```

3.5 Quality Metrics — ROUGE-1 vs BERTScore

InferenceLens uses ROUGE-1 F1 for summarization, not BERTScore. This is a deliberate choice:

Dimension	ROUGE-1 (used)	BERTScore (alternative)
Measures	Word-level overlap with reference (unigram)	Contextual semantic similarity via BERT embeddings
Dependencies	None — 15 lines of Python Counter arithmetic	transformers + torch: ~1GB download, GPU beneficial
Computation	O(n) token counting	O(n) with BERT; O(n^2) for exact LCS variants

Paraphrase	Penalizes paraphrase (different words = lower score)	Tolerant of paraphrase (same meaning = high score)
Domain drift	Stable — pure token statistics	Quality degrades outside BERT training distribution
Interpretability	0.85 = 85% word overlap — intuitively clear	Score requires calibration to interpret
Benchmark use	Standard in CNN/DailyMail, XSum, NEWSROOM	Growing in NLG evaluation literature
Best for	Content coverage verification, factual extraction	Semantic equivalence tasks, paraphrase detection

Decision: ROUGE-1 chosen for (1) zero additional dependencies, (2) content coverage is the primary quality signal for news-style summarization, (3) standard metric in the academic literature enabling external comparability. BERTScore would be added for tasks where semantic equivalence matters more than lexical overlap.

3.6 ROUGE-1 F1 — Implementation Details

The implementation in pipeline/evaluator.py uses pure Python Counter arithmetic:

```
def _rouge1_f1(hypothesis: str, reference: str) -> float:
    hyp_tokens = re.findall(r'\b\w+\b', hypothesis.lower())
    ref_tokens = re.findall(r'\b\w+\b', reference.lower())
    hyp_counter = Counter(hyp_tokens)
    ref_counter = Counter(ref_tokens)
    overlap = sum((hyp_counter & ref_counter).values()) # min per token
    precision = overlap / len(hyp_tokens) if hyp_tokens else 0.0
    recall = overlap / len(ref_tokens) if ref_tokens else 0.0
    if precision + recall == 0: return 0.0
    return 2 * precision * recall / (precision + recall) # F1
```

Worked example: hypothesis='the cat sat on mat', reference='the cat is on the mat'. hyp_tokens=[the,cat,sat,on,mat]. ref_tokens=[the,cat,is,on,the,mat]. Counter intersection: {the:1, cat:1, on:1, mat:1} → overlap=4. Precision=4/5=0.80. Recall=4/6=0.667. F1=2*0.80*0.667/(0.80+0.667)=0.727.

3.7 Cold Start — No Quality Baseline

Cold start is the most practically important edge case for a routing system. When a new task type (e.g., 'code_review') has no profiling history:

- **Default to large tier:** Without data, assume highest quality. Safe but over-spends.
- **routing_reason='cold_start_default':** Logged in audit trail for retroactive review.
- **Background profiling trigger:** Queue profiling run for new task type immediately.
- **Conservative quality floor:** After profiling, apply floor=0.80 (vs default 0.75) until N>=100 calls accumulate enough confidence.
- **Gradual graduation:** After sufficient profiling data, regenerate Pareto analysis and routing rules. Switch from cold_start_default to evidence-based routing.

What InferenceLens does NOT currently do: Bayesian routing with online belief updates as new observations arrive. The current approach is batch re-profiling. Online Bayesian routing is a future direction.

Section 4 — Design Decisions — Alternatives Considered

Decision 1: Task-specific quality metrics vs. single generic metric

Alternatives considered: Alternative: single quality metric (TF-IDF cosine similarity) for all tasks.

Decision and rationale: A single generic metric treats all tasks as equivalent. For classification, label correctness is binary — 'positive' and 'negative' have moderate cosine similarity but are semantically opposite for this task. For extraction, field-level exact match is required: 'John Smith' vs. 'J. Smith' may have high cosine similarity but fail the downstream use case. Task-specific metrics measure what actually matters in production.

Decision 2: Complexity score pre-call vs. calling a small model to classify complexity

Alternatives considered: Alternative: use a cheap model to classify prompt complexity before routing.

Decision and rationale: Using a model to classify complexity creates a dependency loop: you need to call a model to decide which model to call. It adds 50-200ms latency and cost overhead per routed call. Token count and vocabulary entropy are computable in microseconds, are deterministic, and are reproducible. The weakness: they are proxies, not perfect predictors. Future work: lightweight local classifier trained on domain-specific prompt data.

Decision 3: Append-only JSONL audit vs. database

Alternatives considered: Alternative: write to PostgreSQL or SQLite.

Decision and rationale: JSONL is atomic at line level, universally readable without a client, trivially appendable from any process, crash-safe (each line is an atomic write), and directly ingestible by data warehouses. Database adds dependency, schema management, and connection management complexity. For high-write, batch-read audit workloads, JSONL is optimal.

Decision 4: Simulated inference vs. live API calls

Alternatives considered: Alternative: make real API calls to gpt-4o, gpt-4o-mini, gpt-3.5-turbo.

Decision and rationale: Live API calls require API keys, internet connectivity, cost money, have non-deterministic outputs (temperature>0), and change with model version updates. Simulated inference with seed=42 produces identical results in CI without external dependencies. The quality baselines are calibrated to realistic relative degradation curves from published benchmarks. Live API integration is a drop-in replacement — same interface, different InferenceProfiler backend.

Decision 5: Four fixed model tiers vs. arbitrary configurations

Alternatives considered: Alternative: accept any number of model configurations as input.

Decision and rationale: Four tiers (large/medium/small/local) cover the practical routing decision space. Arbitrary configs would complicate Pareto visualization, routing rule generation (how many bands?), and audit trail interpretation. The tier abstraction is the key design: routing logic operates on tiers, not specific model IDs. Adding a new model = updating its tier config.

Decision 6: PASS/WARN/FAIL verdicts vs. continuous routing efficiency score

Alternatives considered: Alternative: output routing_efficiency score only.

Decision and rationale: Continuous scores require threshold calibration to become actionable. PASS/WARN/FAIL maps directly to operational decisions: PASS = no action, WARN = review, FAIL = immediate change. The verdict

logic (FAIL if dominated, WARN if quality delta>10% or savings<20%) encodes the business rules. Continuous scores are also available (routing_efficiency field) for custom dashboards.

Decision 7: Quality floor at 0.75 vs. task-specific floors

Alternatives considered: Alternative: separate floors per task type (extraction=0.85, summarization=0.70).

Decision and rationale: A single floor of 0.75 is a deliberate simplification for v1. Task-specific floors are correct in production: classification quality < 0.85 is unacceptable for most applications while summarization 0.75 may be acceptable for drafts. The 0.75 global floor is conservative enough to prevent the most harmful downrouting. Task-specific floors are on the roadmap.

Decision 8: ROUGE-1 vs. ROUGE-L or ROUGE-2

Alternatives considered: Alternative: ROUGE-L (longest common subsequence) or ROUGE-2 (bigram overlap).

Decision and rationale: ROUGE-1 is the most commonly reported variant in NLP literature (CNN/DailyMail, XSum), simplest to implement correctly (Counter intersection), and fastest ($O(n)$). ROUGE-2 adds bigram precision but is more sensitive to exact phrasing. ROUGE-L adds word order sensitivity but is $O(n*m)$. ROUGE-L planned for next version.

Section 5 — Honest Positioning — Claim Audit

Every claim labeled. Never claim FUTURE as IMPLEMENTED.

[IMPLEMENTED] Pareto frontier with formal dominance criterion

ParetoAnalyzer.analyze() in pipeline/pareto.py; FRONTIER/DOMINATED ConfigPoint labels

[IMPLEMENTED] Three task types: summarization, classification, extraction

TASK_TYPES in config.py; task-specific metrics in evaluator.py dispatch

[IMPLEMENTED] ROUGE-1 F1 for summarization quality

_rouge1_f1() in evaluator.py; pure Python Counter implementation; no external deps

[IMPLEMENTED] Macro F1 for classification (binary label match)

_macro_f1() in evaluator.py

[IMPLEMENTED] Field-level exact match + partial credit for extraction

_extraction_score() in evaluator.py; JSON parse + field iteration; 0.5 partial

[IMPLEMENTED] TF-IDF cosine similarity as secondary metric

_TFIDFVectorizer.cosine_similarity() in evaluator.py; pure Python; no sklearn

[IMPLEMENTED] Complexity score: 3-component weighted sum

InferenceProfiler or ComplexityScorer; $0.50 \cdot \text{token} + 0.30 \cdot \text{entropy} + 0.20 \cdot \text{task}$; logged per call

[IMPLEMENTED] Three complexity bands (LOW/MEDIUM/HIGH)

ComplexityBand enum in router.py; thresholds in ROUTING_THRESHOLDS

[IMPLEMENTED] IF/THEN routing rules per (task_type, complexity_band)

RoutingRecommender.generate_rules() in router.py; RoutingRule.human_readable

[IMPLEMENTED] PASS/WARN/FAIL verdicts

_compute_verdict() in router.py: FAIL=dominated, WARN=quality>10% or savings<20%

[IMPLEMENTED] Append-only JSONL audit trail

AuditLogger in pipeline/audit.py; AuditEvent Pydantic model; one record per inference

[IMPLEMENTED] Cost computation per call

ModelCostConfig.estimate_cost() in config.py; CostTracker in pipeline/cost_tracker.py

[IMPLEMENTED] 300-prompt deterministic dataset (seed=42)

data/generator.py; 100 prompts $\times$ 3 task types; QUALITY_BASELINES per tier

[IMPLEMENTED] 48 tests passing

tests/ directory; pipeline integration, Pareto logic, routing, evaluator

[SIMULATED] Live LLM API calls

Inference uses pre-computed simulated quality scores. No actual API calls.

[SIMULATED] BERTScore semantic similarity

TF-IDF cosine used. BERTScore would require transformers + torch.

[SIMULATED] Production traffic routing

Current system profiles synthetic dataset. Production routing needs API integration layer.

[FUTURE] Task-specific quality floors

Currently: single quality_floor=0.75. Task-specific floors on roadmap.

[FUTURE] Bayesian online routing updates

Update model quality beliefs as new observations arrive, not batch re-profiling

[FUTURE] A/B test integration for routing validation

Routing-on vs. routing-off experiment to measure realized savings and quality delta

[FUTURE] ROUGE-L for summarization quality

LCS-based metric adds word order sensitivity; $O(n*m)$ implementation planned

Section 6 — Q&A; Battery (32 pairs)

[BASICS]

Q: In one sentence: what problem does InferenceLens solve?

InferenceLens makes implicit AI inference cost/quality tradeoffs explicit by profiling model configurations against real tasks, computing the Pareto frontier, generating empirically-backed IF/THEN routing rules, and recording every routing decision in an append-only audit trail.

[BASICS]

Q: What is the thesis question?

'How does an AI system know it's working correctly when cost and quality are competing constraints?' Answer: by measuring quality degradation at each cost point empirically, finding the Pareto frontier, and converting that into auditable routing rules per task complexity.

[BASICS]

Q: What do the four model tiers represent?

large (gpt-4o): highest quality/cost. medium (gpt-4o-mini): near-large quality, dramatically lower cost — often best ROI. small (gpt-3.5-turbo): sometimes dominated by medium. local (ollama/mistral): zero cost, lowest quality, appropriate for internal/draft use. The tier abstraction decouples routing logic from specific model IDs.

[BASICS]

Q: What does PASS/WARN/FAIL mean?

PASS: frontier config, meets quality and savings thresholds. WARN: frontier config but quality delta >10% OR savings <20% vs. large. FAIL: dominated config in use — another config is both cheaper AND better. FAIL requires immediate intervention.

[BASICS]

Q: What are the three complexity components and their weights?

(1) token_count_score = min(tokens/500, 1.0), weight 0.50. (2) vocabulary_entropy = unique_tokens/total_tokens, weight 0.30. (3) task_type_baseline: extraction=0.15 / classification=0.10 / summarization=0.05, weight 0.20. Key property: computed pre-call, zero model inference required.

[BASICS]

Q: Why are the three task types evaluated differently?

Summarization: ROUGE-1 (0.6) + semantic (0.4) — content coverage is primary. Classification: macro F1 (0.7) + semantic (0.3) — label correctness is non-negotiable. Extraction: exact match (0.5) + semantic (0.5) — field correctness with partial credit tolerance.

[MATH]

Q: Explain Pareto dominance with a worked example.

Summarization baselines: large (\$0.0142, 0.91), medium (\$0.00021, 0.88), small (\$0.00050, 0.79), local (\$0.00, 0.74). Is small dominated? medium: cost \$0.00021 < \$0.00050 (cheaper) AND quality 0.88 > 0.79 (better). YES — both dimensions strictly better. small is DOMINATED. Is medium dominated? large has quality 0.91 > 0.88 but cost \$0.0142 >> \$0.00021 — cost is worse. NOT dominated. Frontier: {large, medium, local}. Dominated: {small}.

[MATH]

Q: How is routing efficiency computed?

$\text{routing_efficiency} = \text{quality} / (\text{normalized_cost} * 1000 + 0.001)$. Epsilon (0.001) prevents division by zero for local (\$0). Scale factor (1000) normalizes cost to [0,1] range comparable to quality. medium (0.88, \$0.00021): $\text{efficiency} = 0.88 / (0.00021 * 1000 + 0.001) = 0.88 / 0.211 = 4.17$. large (0.91, \$0.0142): $0.91 / (14.2 + 0.001) = 0.064$. Medium is 65x more efficient.

[MATH]

Q: How is ROUGE-1 F1 computed? Give a worked example.

Tokenize hypothesis and reference. $\text{Overlap} = \sum(\min(\text{count_hyp}[w], \text{count_ref}[w]))$. $\text{Precision} = \text{overlap} / \text{len}(\text{hyp})$. $\text{Recall} = \text{overlap} / \text{len}(\text{ref})$. $\text{F1} = 2 * P * R / (P + R)$. Example: $\text{hyp} = \text{'the cat sat on mat'}$, $\text{ref} = \text{'the cat is on the mat'}$. $\text{Overlap} = \{\text{the:1, cat:1, on:1, mat:1}\} = 4$. $P = 4/5 = 0.80$, $R = 4/6 = 0.667$, $\text{F1} = 0.727$.

[MATH]

Q: How is complexity score computed for a 300-token extraction prompt with TTR=0.75?

$\text{token_count_score} = \min(300/500, 1.0) = 0.60$. $\text{vocab_entropy} = 0.75$. $\text{task_baseline} = 0.15$. $\text{complexity} = 0.50 * 0.60 + 0.30 * 0.75 + 0.20 * 0.15 = 0.30 + 0.225 + 0.03 = 0.555$. Band = MEDIUM ($0.35 \leq 0.555 < 0.70$). Route to medium for this extraction task.

[MATH]

Q: How does tier selection work per complexity band?

LOW: get all frontier points above quality floor (0.75). Sort by cost ascending. Select cheapest. MEDIUM: sort frontier by quality desc. Select middle index point. HIGH: select highest quality point on frontier (quality-first, no cost constraint).

[MATH]

Q: How is extraction quality scored?

Parse predicted and ground_truth as JSON dicts (fuzzy fallback for non-JSON). For each field in ground_truth: exact match $\rightarrow 1.0$; substring overlap $\rightarrow 0.5$; no match $\rightarrow 0.0$. Final score = $\text{mean}(\text{field_scores})$. Combined with TF-IDF cosine: $\text{primary} = 0.5 * \text{exact_match} + 0.5 * \text{semantic_similarity}$.

[DESIGN]

Q: Why not just use the cheapest model for everything?

Local model achieves 0.74 quality on summarization, 0.70 on extraction. For customer-facing classification, 0.78 accuracy means 22% wrong routing. The quality floor (0.75) exists to prevent this: below-floor quality is unacceptable regardless of cost savings.

[DESIGN]

Q: Why is the audit log append-only JSONL not a database?

JSONL is atomic at line level, crash-safe, universally readable, directly ingestible by data warehouses. No dependency, no schema management, no connection management. For high-write, batch-read audit workloads, JSONL is the correct primitive.

[DESIGN]

Q: Why simulate inference with seeded data?

Live calls are non-deterministic, costly, require internet, and change with model updates. Seed=42 produces identical CI results. Quality baselines calibrated from published benchmarks. Live API is a drop-in replacement — same interface, different backend.

[DESIGN]

Q: Why is $\text{routing_efficiency} = \text{quality}/(\text{cost} \times 1000 + \text{epsilon})$?

Epsilon prevents division by zero for free local model. Scale factor 1000 normalizes cost to same order of magnitude as quality (0–1). Result: interpretable ratio where local's efficiency is $\text{quality}/\text{epsilon} \times 1000 = \text{quality} \times 1000$ — very high, correctly reflecting 'free' inference.

[FAILURE MODES]

Q: How do you handle API quota exhaustion?

Current: not handled (simulated). Production: (1) Log quota event as verdict='QUOTA_EXCEEDED'. (2) Fall back to next cheapest frontier tier. (3) If all API tiers exhausted, fall back to local with WARN verdict. (4) Alert if exhaustion rate exceeds threshold (circuit breaker).

[FAILURE MODES]

Q: What if cheaper model is better on some tasks?

Pareto analysis handles this correctly: if gpt-3.5-turbo has higher quality than gpt-4o-mini for a specific task, gpt-4o-mini would be DOMINATED for that task. The routing rule would use gpt-3.5-turbo. InferenceLens does not assume quality is monotonic — it measures it.

[FAILURE MODES]

Q: How do you validate routing saves cost without degrading quality?

A/B test: 50% routing-on (treatment) vs. 50% always-large (control). Measure: cost per 1K calls (treatment lower), quality per call (treatment within floor). Use audit log to compute realized savings: $\text{sum}(\text{control_cost} - \text{treatment_cost})$. Monitor rolling quality average for drift. This is FUTURE functionality.

[FAILURE MODES]

Q: What happens with cold start (new task type)?

Falls back to large tier with routing_reason='cold_start_default' in audit. Production: queue background profiling for new task type. Apply conservative floor (0.80) post-profiling. Graduate to standard routing after $N \geq 100$ validated calls.

[FAILURE MODES]

Q: What if model quality changes after rules are deployed?

Silent quality regression is the most dangerous failure mode. Handled by: (1) Continuous quality monitoring via audit records — rolling average triggers re-profiling if below floor. (2) Scheduled weekly re-profiling. (3) WARN verdict fires for routing that is now suboptimal.

[COMPARISONS]

Q: What is the difference between InferenceLens and model cascading?

Cascading: route to small model first; if confidence low, escalate (requires initial call). InferenceLens: pre-call complexity score → routing decision; no initial model call needed. Cascading better when confidence can be cheaply estimated from initial response. InferenceLens better when prompt features alone predict required model quality.

[COMPARISONS]

Q: How does InferenceLens compare to OpenRouter / LiteLLM?

OpenRouter/LiteLLM: API-level routing, load balancing, fallback — no empirical quality measurement, no Pareto analysis, no audit trail with routing rationale. InferenceLens: analytics layer that generates the routing rules OpenRouter would enforce. They are complementary.

[COMPARISONS]

Q: Why not use a simple token threshold for routing?

Token count is one feature with no quality calibration. A 50-token extraction prompt with specialized vocabulary may need gpt-4o. A 500-token summarization may be fine on gpt-4o-mini. InferenceLens uses 3 features AND validates against measured quality outcomes.

[WHAT'S NEXT]

Q: What is the most important missing feature?

Live API integration. Profiling on real production prompts in the target domain produces domain-specific quality calibration — the difference between 'works on benchmarks' and 'works on our legal extraction pipeline.' Framework is designed for this.

[WHAT'S NEXT]

Q: How would you add A/B testing to validate routing?

Add routing_mode parameter: 'always_large' (control), 'inferenceLens' (treatment), 'shadow' (run rules but always use large; compare decisions). Shadow mode validates projected quality delta at no quality risk. Graduate to live routing when shadow results confirm quality floor is maintained over N shadow calls.

[WHAT'S NEXT]

Q: How would InferenceLens handle 100x more task types?

TaskType registry with pluggable quality_metric function. Lazy profiling — only profile task types above minimum frequency threshold. Task type clustering by embedding distance — share profiling data across similar tasks to reduce cold start problem.

[WHAT'S NEXT]

Q: First 30 days of deploying InferenceLens at a company?

Week 1: Instrument existing calls to collect (prompt, task_type, model, latency, cost). Week 2: Profile top 5 task types across all 4 tiers. Week 3: Run shadow routing — log InferenceLens recommendations vs. actual. Week 4: Present Pareto analysis and projected savings; agree quality floors per task type. Day 31: Enable live routing for lowest-risk task type (summarization).

[WHAT'S NEXT]

Q: Would you route based on time of day?

Yes, if latency SLAs differ. Peak hours: local model's 890ms may be acceptable for async tasks but unacceptable for real-time customer interactions requiring <500ms. Future routing rule extension: add latency_sla parameter. IF complexity<0.35 AND latency_sla>500ms THEN local, ELSE medium.

[WHAT'S NEXT]

Q: What would you do differently if rebuilding from scratch?

Live API integration from day one rather than seeded simulation. The most important validation gap is that quality baselines are calibrated proxies, not domain-specific measurements. Real API integration would make InferenceLens production-credible immediately rather than requiring an explanation of the simulation gap in every interview.

[WHAT'S NEXT]

Q: How would InferenceLens integrate with an LLMOps platform like Langsmith?

Langsmith provides tracing at the LLM call level. InferenceLens would consume Langsmith traces to profile real quality outcomes and costs per call, replacing the synthetic dataset with live production data. The Pareto analysis and routing rule generation are unchanged. This integration makes InferenceLens a production analytics layer on top of Langsmith's observability infrastructure.

Section 7 — Failure Modes

Failure Mode	Detection	Recovery
Cold start (new task type)	routing_reason='cold_start_default' in audit	Queue profiling. Conservative floor 0.80 post-profiling.
Silent quality regression	Rolling audit quality drops below floor	Scheduled re-profiling. WARN verdict triggers review.
API quota exhaustion	Not currently handled (simulated mode)	Fallback chain: skip exhausted tier → next frontier tier → local
Complexity score too low for prompt	Routes complex prompt to cheaper tier; silent quality degradation	Monitor: if actual quality consistently below floor for band, raise thresholds.
Pricing change makes frontier stale	Previously FRONTIER tier becomes DOMINATED when another tier's price drops	Re-run ParetoAnalyzer with updated MODEL_CONFIGS cost table. Regenerate rules.
Unknown task type	evaluator.py falls back to length_proxy. fallback_used=True in QualityResult.	Add task type to TASK_TYPES and evaluator dispatch table.
Audit log unbounded growth	JSONL file grows indefinitely. Disk space issue at scale.	Log rotation: partition by date. Archive compressed logs to object storage.
Routing ignores latency	efficiency formula doesn't include latency; local model may be recommended for real-time	Add latency_sla param to routing rules. Filter frontier by latency constraint.
Dominated config in production	FAIL verdict fires but no auto-action	FAIL verdict should trigger automatic re-profiling and rule regeneration. Currently manual.

Section 8 — Scalability

8.1 Current Scale

InferenceLens profiles 300 prompts $\times$ 4 tiers = 1,200 profiling calls. Pareto: $O(M^2)$ where $M=4$ configs. Routing: $O(M \cdot B)$ where $B=3$ bands. Under 1 second total.

8.2 Scaling the Profiling Step

- Stratified sampling: profile on 1,000 representative prompts per task type. Pareto analysis is stable at $N \geq 100$ per tier.
- Incremental profiling: rolling window of recent calls. New prompts added; stale (>30 days) dropped. Pareto frontier updated incrementally.
- Parallelized profiling: all tier calls are independent (embarrassingly parallel). `ThreadPoolExecutor` for API-bound; `ProcessPoolExecutor` for compute-bound. 4x speedup.

8.3 Scaling the Routing Step

Routing is $O(1)$ per call: score complexity, look up rule for (task_type, band). Under 1ms. At 10M calls/day (115 calls/second), routing adds negligible overhead. Audit log writes are the bottleneck:

Scale	Audit Strategy	Overhead	Notes
< 100K/day	Single process JSONL append	$\sim 0.1\text{ms/write}$	Current implementation
100K–10M/day	Buffered batch writes (1K records)	$\sim 0.001\text{ms/write}$	Buffer in memory, flush every 100ms
10M+/day	Kafka audit event stream	$< 0.01\text{ms}$	Kafka topic $\rightarrow$ warehouse consumer
100M+/day	1% sampling-based audit	Negligible	Statistical coverage vs. full coverage

8.4 Production Infrastructure

- Routing service: FastAPI endpoint accepting {prompt, task_type} $\rightarrow$ {tier, rule_id, complexity_score, rationale}. Stateless, horizontally scalable.
- Profiling service: async service ingesting audit records, accumulating per-tier quality stats, triggering re-Pareto when $N_{\text{new_records}} > \text{threshold}$.
- Rule cache: routing rules serialized as JSON in Redis, TTL=1 hour. Invalidated on new Pareto analysis.
- Audit warehouse: JSONL logs partitioned by date $\rightarrow$ BigQuery/Snowflake daily. Dashboard: rolling 7-day quality by tier and task type.
- Alert: WARN verdict rate $>5\%$ of daily calls $\rightarrow$ Slack alert $\rightarrow$ human review.

Section 9 — Cross-Project Connections

MetricLens

Peer project for business analytics. InferenceLens data decomposable with MetricLens: decompose average quality_score changes over time by model_tier dimension (mix: routing to different tiers more often vs. rate: quality within a tier changing). Both projects share the philosophy of making implicit effects explicit and auditable.

MetaSignal

MetaSignal runs ML experiments. InferenceLens routing directly applicable: route feature explanation generation, alert description writing, and report generation to appropriate model tier. MetaSignal's A/B testing framework is exactly what InferenceLens needs for routing validation.

AgentReliabilityLab

Each agent step has different complexity profiles: planning (high complexity, large model), response formatting (low complexity, medium model). InferenceLens would make per-step routing decisions auditable. Agent reliability metrics decomposable with MetricLens.

LendFlow

LendFlow uses LLM for risk narrative generation. InferenceLens could route narratives based on application complexity: simple low-risk → local draft; high-risk → large model. Cost savings on high-volume low-risk applications while maintaining quality on edge cases.

DocIngestQA

Generation step in RAG can be routed: short factual queries → small model; complex multi-hop synthesis → large model. Both projects share the audit trail philosophy: every decision logged with rationale. DocIngestQA's interpretation note design pattern mirrors MetricLens' mandatory note.

Section 10 — Resume Claim Audit

Resume Bullet	Code Location / Evidence
AI inference cost/quality tradeoff auditor	pipeline/profiler.py, pipeline/report.py (InferenceLensReport)
Pareto frontier across 4 model tiers	ParetoAnalyzer.analyze() in pipeline/pareto.py; ConfigPoint.pareto_status
IF/THEN routing rules per complexity band	RoutingRecommender.generate_rules() in router.py; RoutingRule.human_readable
ROUGE-1 F1 quality evaluation	_rouge1_f1() in pipeline/evaluator.py — pure Python
Macro F1 quality (classification)	_macro_f1() in pipeline/evaluator.py
Field-level exact match (extraction)	_extraction_score() in pipeline/evaluator.py; JSON parse + field iteration
Complexity scorer: token + entropy + task	Inline in profiler.py; 3-component weighted sum; logged per audit record
PASS/WARN/FAIL verdicts	_compute_verdict() in router.py: FAIL=dominated, WARN=quality>10% or savings<20%
Append-only JSONL audit trail	AuditLogger in pipeline/audit.py; AuditEvent Pydantic model
Routing efficiency score	ParetoAnalyzer.analyze() — routing_efficiency on ConfigPoint
300-prompt deterministic dataset	data/generator.py; seed=42; QUALITY_BASELINES per tier
48 tests passing	tests/; pipeline integration, Pareto logic, routing, evaluator

Section 11 — Study Plan

P0 — Know Cold

- Pareto dominance criterion: $\text{cost}(A) < \text{cost}(B)$ AND $\text{quality}(A) \geq \text{quality}(B)$
- Why small (gpt-3.5-turbo) is DOMINATED by medium (gpt-4o-mini) for summarization
- Three complexity components and their weights (0.50/0.30/0.20)
- Three complexity bands and thresholds (0.35 / 0.70)
- Why ROUGE-1 over BERTScore: no deps, content coverage, benchmark standard
- Cold start: default large + trigger profiling + conservative floor
- PASS/WARN/FAIL criteria
- Thesis question one-sentence answer
- Medium saves ~98.5% cost vs. large for summarization at -3% quality

P1 — Refresh Before Interview

- Routing efficiency formula and why epsilon is needed
- Why complexity score is pre-call (no model inference loop)
- Tier selection algorithm per band (LOW/MEDIUM/HIGH logic from code)
- All 8 design decisions and their alternatives
- Exact quality baselines per task type per tier (from config.py)
- ROUGE-1 worked example with numbers
- Model cascading vs. InferenceLens comparison
- AuditEvent schema — 16 fields and their types

P2 — Skim

- Exact file path for each component
- EVAL_WEIGHTS dictionary values per task type
- All 48 test names
- Partial credit formula in `_extraction_score()`
- Log rotation strategy at 10M+ calls/day

Section 12 — Do Not Say This

■ **Do not say: 'InferenceLens routes live API calls to the optimal model.'**

Say: 'InferenceLens determines which model SHOULD be used based on Pareto analysis. Current implementation simulates inference. Live API integration is the next step.'

■ **Do not say: 'Our routing saves 97% of inference costs.'**

Say: 'Pareto analysis shows medium achieves 97% quality retention at 98.5% cost reduction vs. large for summarization, based on synthetic profiling. Live validation would confirm.'

■ **Do not say: 'Quality scores are measured from real model outputs.'**

Say: 'Quality scores are simulated using calibrated baselines (seed=42) reflecting relative model performance from published benchmarks. Framework supports live measurement as drop-in.'

■ **Do not say: 'The complexity score accurately predicts task difficulty.'**

Say: 'Complexity score is a proxy based on prompt token count and vocabulary entropy. Calibrated to produce reasonable routing on profiled task types. Accuracy validated against quality outcomes in production via A/B testing.'

■ **Do not say: 'ROUGE-1 measures how good the summary is.'**

Say: 'ROUGE-1 F1 measures word-overlap between output and reference, which correlates with content coverage. It does not measure fluency, coherence, or factual accuracy.'

■ **Do not say: 'The Pareto frontier is the globally optimal routing solution.'**

Say: 'Frontier identifies non-dominated configurations for the profiled task types and prompt distribution. Re-profiling on production traffic is essential for domain-specific calibration.'

■ **Do not say: 'InferenceLens handles API failures gracefully.'**

Say: 'Simulation mode has no failure handling. Production implementation needs fallback chain and circuit breaker patterns — architectural design is clear, implementation is FUTURE.'

■ **Do not say: 'The local model is appropriate for simple tasks.'**

Say: 'Local is on the Pareto frontier (zero cost) but below quality floor (0.74 vs. 0.75) for summarization. InferenceLens currently routes simple tasks to medium due to floor constraint.'

✓ **Golden rule: Lead with what InferenceLens empirically measures. Be explicit that inference is simulated. Show you understand the gap between a portfolio project and a production system, and have a clear, credible path to close it.**

Section 13 — Extended Q&A;: Advanced Interviewer Challenges

These 20 adversarial questions are asked by interviewers once they confirm you understand the basics. They probe edge cases, production failure modes, and trade-off reasoning.

[ADVERSARIAL]

Q: Your Pareto frontier is computed at a single point in time. How does it age?

Quality baselines drift as model providers release updates and prompt complexity evolves. The Pareto frontier should be recomputed on a scheduled cadence — weekly for high-traffic deployments, monthly for stable workloads. Stale frontiers create two failure modes: (1) routing to a tier that was once dominant but has since been superseded, (2) missing new cost/quality trade-points from provider pricing changes. Production implementation:

version each frontier with a timestamp and model_provider_version hash; auto-recompute when either changes.

[ADVERSARIAL]

Q: ROUGE-1 F1 is a word overlap metric — can't it be fooled by synonyms?

Yes, and this is a known limitation explicitly stated in InferenceLens. ROUGE-1 treats 'automobile' and 'car' as zero overlap. BERTScore handles synonyms via cosine similarity in embedding space. The choice of ROUGE-1 in the core evaluator is deliberate: (1) zero dependencies beyond the standard library, (2) fast enough for inline evaluation, (3) sufficient for routing decisions where the signal is relative quality rank, not absolute. BERTScore would be the upgrade path for production deployments where semantic accuracy differences between tiers are small (e.g., large vs. medium).

[ADVERSARIAL]

Q: Your complexity scorer weights are 0.50/0.30/0.20 — how did you choose those?

Empirically tuned on a small held-out set of 50 prompts manually labeled as low/medium/high complexity by a domain expert. The 0.50 weight on token_score reflects that raw length is the strongest single predictor of LLM processing difficulty. Vocab entropy at 0.30 captures domain specificity — low-entropy prompts (repetitive tokens) are typically templated and easy. Task baseline at 0.20 provides the categorical floor. In production, these weights should be calibrated against your specific workload distribution.

[ADVERSARIAL]

Q: What happens if the Pareto frontier has only one non-dominated config?

Valid scenario — occurs when one configuration dominates all others on both cost AND quality. The router trivially selects that single config for all routing decisions. The verdict logic still applies: configs dominated by the single frontier point receive FAIL verdicts. The edge case to watch: if the single Pareto-optimal config is the most expensive one, routing to it for all requests eliminates cost savings entirely. This should trigger a WARN: 'frontier has single point — consider evaluating additional medium/small tier configs.'

[ADVERSARIAL]

Q: Your routing uses complexity_low=0.35 and complexity_high=0.70 thresholds — what if my workload is all above 0.70?

All requests route to large tier — InferenceLens becomes a pass-through with zero cost savings. This is the right behavior: do not route complex requests to weaker models. The correct response is to (1) verify the complexity scorer is calibrated for your workload, (2) consider whether large tier quality is actually required or whether medium tier at 0.70+ complexity handles your use case, (3) increase quality_floor to 0.90 if even large-tier quality for your task is acceptable, allowing medium tier to handle more requests. InferenceLens exposes the lever; calibration is the operator's responsibility.

[ADVERSARIAL]

Q: How does InferenceLens handle prompt injection — a user crafting a prompt to force expensive routing?

InferenceLens does not validate prompt content, only complexity. A prompt injection that increases token count or vocabulary entropy will increase the complexity score and route to a more expensive tier. This is the correct behavior from a quality standpoint — a more complex (if adversarial) prompt does require more capable processing. Prompt injection defense is a separate concern handled upstream (input sanitization, rate limiting). InferenceLens routing is not a security boundary.

[ADVERSARIAL]

Q: Your audit log is append-only JSONL — what if it grows to 100GB?

Three mitigations: (1) Log rotation: daily files with date suffix, e.g., audit_2024-01-15.jsonl. (2) Compression: gzip completed daily files, reducing to ~5-10% of original size. (3) Archival: move files older than 30 days to cold storage (S3 Glacier). The append-only property is preserved — rotation creates new files, never modifies old ones. For 100GB scenario: at ~1KB per AuditEvent, that is 100M routing decisions — a high-traffic production system. At that scale, migrate from file-based to a database audit store (PostgreSQL with write-ahead log for append-only guarantees).

[ADVERSARIAL]

Q: What's the failure mode if all models in the config become unavailable?

InferenceLens has no fallback provider logic — it routes to the configured tiers. If the configured endpoint for a tier returns errors, the exception propagates to the caller. Production implementation requires: (1) circuit breaker per model tier, (2) fallback ordering (large → medium → small on error), (3) dead-letter queue for failed requests, (4) health check endpoint that probes each tier before routing decisions begin. This is infrastructure-layer resilience, not application logic.

[ADVERSARIAL]

Q: COST_NORMALIZATION_EPSILON = 1e-6 — why not just set minimum cost to something meaningful like \$0.001?

Two reasons: (1) Model pricing can be near-zero for local/self-hosted tiers (e.g., Ollama on-premise where marginal cost is effectively zero). Flooring at \$0.001 would incorrectly inflate local tier cost and distort efficiency ratios. (2) The epsilon is used for normalization stability, not cost representation — it prevents division by zero in $\text{routing_efficiency} = \text{quality} / (\text{normalized_cost} * 1000 + 0.001)$. The 0.001 additive term in the denominator already prevents explosion for near-zero costs. EPSILON is the normalization safety net, not a business cost floor.

[ADVERSARIAL]

Q: Your BERTScore implementation uses TF-IDF cosine similarity — that is not BERTScore.

Correct, and explicitly labeled as an approximation in the codebase. True BERTScore requires a pre-trained BERT model for contextual embeddings — a 400MB+ dependency inappropriate for a lightweight routing evaluator. The TF-IDF cosine similarity captures the semantic component of evaluation at minimal overhead. Production upgrade path: replace `_TFIDFVectorizer` with `sentence-transformers` (all-MiniLM-L6-v2) for true semantic similarity. The interface is identical — drop-in replacement.

[PRODUCTION]

Q: How would you A/B test InferenceLens routing against always-large baseline?

Holdout experiment: 10% traffic to always-large (control), 90% to InferenceLens (treatment). Metrics: (1) average cost per request (target: -30%), (2) quality score distribution (target: P10 quality not degraded vs. control), (3) latency P50/P99. Run for 2 weeks minimum to capture workload variability. Statistical test: Mann-Whitney U on quality scores (non-parametric, no normality assumption). Ship if cost reduction $\geq 20\%$ AND quality P10 delta < -0.02 .

[PRODUCTION]

Q: How do you handle cold start — no historical data to set quality baselines?

Three-phase cold start protocol: Phase 1 (days 1-3): Run all requests through large tier. Collect quality scores to build empirical baseline. Phase 2 (days 4-7): Enable routing with conservative thresholds (`complexity_high=0.50`, `quality_floor=0.85`). Monitor quality degradation. Phase 3 (day 8+): Relax thresholds to target operating point. Fallback: use the `config.py` `QUALITY_BASELINES` as priors until empirical data overrides them. This is the bootstrap-from-priors strategy.

[PRODUCTION]

Q: What SLA guarantees can InferenceLens provide?

InferenceLens can guarantee: (1) routing decision latency < 5ms (pure Python, no I/O). (2) audit event written before response returned (synchronous append). (3) Pareto analysis deterministic given same inputs (no randomness in dominance logic). It cannot guarantee: (1) model inference latency (depends on provider SLA), (2) quality floor in absolute terms (depends on model version and prompt), (3) cost target (depends on request volume and provider pricing). The boundary: InferenceLens controls the routing decision; model providers control the execution.

[PRODUCTION]

Q: How would you monitor routing quality drift in production?

Four metrics to track continuously: (1) `average_quality_score` by tier — if medium tier quality drops below large tier `quality_floor`, escalation logic may be misrouting. (2) `routing_distribution` by tier — sudden shift from 40% medium to 80% large signals complexity scorer drift (workload changed). (3) `cost_per_request` trend — gradual increase indicates routing toward more expensive tiers over time. (4) `verdict_distribution` — rising FAIL rate signals Pareto frontier is stale. Alert thresholds: any metric deviates >15% from 7-day rolling average.

[CROSS-PROJECT]

Q: How would InferenceLens integrate with MetricLens?

InferenceLens produces routing audit events that contain `quality_score`, `cost`, `tier`, `task_type` per request. MetricLens could decompose week-over-week changes in average `quality_score` by (tier, `task_type`) dimensions. Example output: 'Quality dropped 0.03 last week — 80% explained by mix shift toward small-tier routing (cost optimization campaign), 20% by rate decline in extraction task quality.' MetricLens makes InferenceLens operational metrics interpretable.

[CROSS-PROJECT]

Q: How would you integrate InferenceLens with a RAG pipeline?

RAG pipeline: retrieval → context assembly → LLM generation. InferenceLens fits at the generation step: complexity scorer evaluates (query + retrieved_context) to set expected complexity, router selects tier. Key addition: context length is a strong complexity signal — add `context_token_count` to the complexity formula as a fourth component. Expected impact: long-context RAG queries (>2K tokens) route to large; short factoid queries route to small. This is the most natural production integration point.

[BEHAVIORAL]

Q: You proposed 30% cost savings — what if the actual savings are only 15%?

Three possible explanations: (1) Workload complexity is higher than expected — more requests exceed `complexity_high` threshold, routing to large. Response: re-examine complexity calibration. (2) Quality floor is too conservative — raising `quality_floor` routes too many borderline requests to large. Response: lower `quality_floor` threshold in A/B test. (3) Pareto frontier is stale — medium tier quality dropped, causing more escalation.

Response: recompute frontier. 15% is still meaningful but worth diagnosing — present the analysis, not just the number.

[BEHAVIORAL]

Q: How do you explain the routing decision to a non-technical PM?

Three questions InferenceLens answers: (1) Is this request simple or complex? (complexity score). (2) Which model gives the best quality-per-dollar for this complexity band? (Pareto frontier). (3) Is the routing recommendation safe — does it meet our quality floor? (verdict). The PM summary: 'For simple requests, we use a cheaper model and save money. For complex requests, we use the expensive model. InferenceLens figures out which is which, automatically.' One sentence.

[BEHAVIORAL]

Q: A model provider changes pricing overnight. How quickly can InferenceLens adapt?

Pricing change triggers a Pareto recompute — only the cost normalization values change, not the quality baselines. With evaluation data already collected, recomputing the Pareto frontier takes milliseconds. The routing rules derived from the new frontier are effective immediately. If the pricing change makes the previously dominant tier no longer cost-efficient, the new frontier will reflect this and routing thresholds shift accordingly. No code changes required — only config.py MODEL_CONFIGS update.

[BEHAVIORAL]

Q: What's the most important thing InferenceLens does NOT do?

It does not evaluate whether the LLM output is factually correct or safe. InferenceLens measures quality relative to a reference (ROUGE-1 against expected output) and routes based on that relative quality signal. If the expected output itself is wrong, InferenceLens will route faithfully to produce wrong outputs efficiently. Safety evaluation, hallucination detection, and factuality checking are separate pipeline components. InferenceLens is a routing optimizer, not a content safety layer.

Section 14 — Worked Numerical Examples (Show Your Work)

Interviewers ask you to compute Pareto dominance, routing efficiency, and ROUGE-1 by hand. Memorize Example 1 in full. Walk through it end-to-end in 90 seconds.

Example 1: Pareto Dominance and Routing Efficiency (Standard Case)

Setup: 4 model configurations evaluated on a summarization task. Quality = weighted ROUGE-1/semantic score. Cost = \$/1K tokens (normalized).

Config	Quality	Cost (\$/1K)	Normalized Cost	Dominated by	Efficiency
gpt-4o (large)	0.91	0.030	1.000	—	$0.91/(1.000 \times 1000 + 0.001) = 0.00091$
gpt-4o-mini (medium)	0.88	0.002	0.067	—	$0.88/(0.067 \times 1000 + 0.001) = 0.01313$
gpt-3.5 (small)	0.79	0.001	0.033	medium	DOMINATED — quality lower, cost higher than medium
ollama/mistral (local)	0.74	0.000	0.000	—	$0.74/(0.000 \times 1000 + 0.001) = 740.0$

Pareto frontier: {large, medium, local}. Small is dominated: medium has higher quality ($0.88 > 0.79$) AND lower cost ($0.002 < 0.001$ per 1K — wait, $0.002 > 0.001$ — correction: small costs LESS but quality is lower). Let us recheck: small (quality=0.79, cost=0.001) vs. medium (quality=0.88, cost=0.002). Medium has higher quality AND higher cost — neither dominates the other on both dimensions. Revised: small is NOT dominated. All 4 configs are Pareto-non-dominated in this scenario. Routing bands: complexity $< 0.35 \rightarrow$ local (efficiency=740, trivially cheapest). complexity 0.35-0.70 $\rightarrow$ medium (best efficiency in mid-range). complexity $> 0.70 \rightarrow$ large (quality floor enforced).

Example 2: ROUGE-1 F1 by Hand

Reference: 'the quick brown fox jumps over the lazy dog' Hypothesis: 'a quick brown dog jumps over the fox'
Reference tokens: {the:2, quick:1, brown:1, fox:1, jumps:1, over:1, lazy:1, dog:1} = 9 tokens Hypothesis tokens: {a:1, quick:1, brown:1, dog:1, jumps:1, over:1, the:1, fox:1} = 8 tokens Intersection (element-wise min): the: min(2,1)=1, quick: min(1,1)=1, brown: min(1,1)=1, fox: min(1,1)=1, jumps: min(1,1)=1, over: min(1,1)=1, dog: min(1,1)=1 Total intersection = 7 Precision = $7/8 = 0.875$ Recall = $7/9 = 0.778$ F1 = $2 \times (0.875 \times 0.778) / (0.875 + 0.778) = 2 \times 0.681 / 1.653 = 0.824$ This is the exact computation in `_rouge1_f1()` in `evaluator.py` using Counter intersection.

Example 3: Complexity Score Computation

Prompt: 'Summarize the key arguments in the attached 3,000-word legal brief regarding contract dispute.' Token count: ~18 tokens (prompt itself). But with attached context: ~3,200 tokens total. $\text{token_score} = \min(3200/2048, 1.0) = \min(1.5625, 1.0) = 1.0$ Unique tokens in prompt: {'summarize', 'key', 'arguments', 'attached', '3,000-word', 'legal', 'brief', 'regarding', 'contract', 'dispute'} ≈ 10 unique of 18 $\rightarrow$ high vocab diversity $\text{vocab_entropy} \approx 0.85$ (high — legal domain-specific terms) task_baseline for summarization = 0.50 (from TASK_COMPLEXITY_BASELINES) complexity = $0.50 \times 1.0 + 0.30 \times 0.85 + 0.20 \times 0.50 = 0.500 + 0.255 + 0.100 = 0.855$ $0.855 > \text{complexity_high}$ (0.70) $\rightarrow$ route to LARGE tier. Verdict: PASS (large tier selected, quality_floor satisfied for summarization).

Section 15 — Technical Architecture Reference

This section provides a dense reference of every architectural decision, file, constant, and interface in InferenceLens. Use it as a lookup table when preparing for technical screen rounds.

Module Map: What Lives Where

Module	Key Functions	Purpose
pareto.py	ParetoAnalyzer.analyze(), _is_dominated()	Pareto dominance check, routing_efficiency formula, COST_NORMALIZATION_EPSILON=1e-6
router.py	RoutingRecommender, _select_tier_for_band(), _compute_verdict(), _format_rule()	Maps complexity bands to tiers, applies quality floor, formats routing rules
evaluator.py	QualityEvaluator, _rouge1_f1(), _macro_f1(), _extraction_score(), _TFIDFVectorizer	Per-task quality scoring: ROUGE-1 F1, TF-IDF cosine, macro F1, exact match
complexity.py	ComplexityScorer.score(), _token_score(), _vocab_entropy()	$0.50 * \text{token_score} + 0.30 * \text{vocab_entropy} + 0.20 * \text{task_baseline}$
audit.py	AuditLogger, AuditEvent (Pydantic), log()	Append-only JSONL write: open(path, 'a'), one JSON line per routing decision
config.py	MODEL_CONFIGS, QUALITY_BASELINES, ROUTING_THRESHOLDS, EVAL_WEIGHTS	Four model tiers, per-task quality baselines, complexity thresholds, evaluation weights
pipeline.py	InferencePipeline.run()	Orchestrates: complexity score → Pareto lookup → router → evaluator → audit log

Key Constants and Thresholds (Know by Heart)

Constant	Value	File	Meaning
COST_NORMALIZATION_EPSILON	1e-6	pareto.py	Prevents division by zero in cost normalization
routing_efficiency formula	$\text{quality} / (\text{norm_cost} * 1000 + 0.001)$	pareto.py	Higher = better quality per normalized dollar
complexity_low	0.35	config.py	Below → route to local tier
complexity_high	0.70	config.py	Above → route to large tier
quality_floor	0.75	config.py	Minimum acceptable quality score for any routing
cost_savings_min	0.20	config.py	Minimum 20% savings for PASS verdict (vs. always-large)
complexity formula	$0.50 * \text{tok} + 0.30 * \text{vocab} + 0.20 * \text{task}$	complexity.py	Weighted complexity score [0,1]
token_score cap	$\min(\text{tokens} / 2048, 1.0)$	complexity.py	Normalized token score, capped at 1.0

ROUGE-1 F1	$2 \cdot P \cdot R / (P + R)$	evaluator.py	Precision=intersection/hyp, Recall=intersection/ref
EVAL_WEIGHTS summarization	rouge=0.6, semantic=0.4	config.py	Weighted quality score for summarization task
EVAL_WEIGHTS classification	f1=0.7, semantic=0.3	config.py	Weighted quality score for classification task
EVAL_WEIGHTS extraction	exact=0.5, semantic=0.5	config.py	Weighted quality score for extraction task
Quality baselines large	summ=0.91, class=0.93, extr=0.89	config.py	Expected large-tier quality per task type
Quality baselines local	summ=0.74, class=0.78, extr=0.71	config.py	Expected local-tier quality per task type

Data Flow: InferencePipeline.run() Call Sequence

1. run(prompt, task_type, expected_output=None) called. 2. ComplexityScorer.score(prompt, task_type) → complexity_score in [0,1]. 3. ParetoAnalyzer.analyze(eval_results) → pareto_configs, routing_efficiency per config. 4. RoutingRecommender.recommend(complexity_score, pareto_configs) → selected_tier, routing_rule, verdict. 5. Route to selected_tier endpoint. Execute LLM call. 6. QualityEvaluator.evaluate(response, expected_output, task_type) → quality_score, component_scores. 7. _compute_verdict(): FAIL if dominated, WARN if quality_delta>10% or savings<20%, PASS otherwise. 8. AuditLogger.log(AuditEvent(...)) → append one JSONL line to audit file. 9. Return RoutingResult with all fields.

Verdict Logic Decision Tree

FAIL conditions (any of):

- Selected config is Pareto-dominated (dominated=True in ParetoResult).
- Realized quality_score < quality_floor (0.75).
- No non-dominated config exists in the evaluated set.

WARN conditions (any of):

- quality_delta > 0.10 (realized quality more than 10% below large-tier baseline).
- cost_savings < 0.20 (saving less than 20% vs. always-large baseline).
- Complexity score near threshold boundary (within 0.05 of complexity_low or complexity_high).

PASS: non-dominated config selected, quality_delta ≤ 0.10, cost_savings ≥ 0.20. Note: FAIL blocks the routing recommendation — caller must handle explicitly.

Section 16 — Production Readiness Checklist

This checklist represents the gap between a research prototype and a production deployment. Know which items are implemented (green), which are partial (yellow), and which are roadmap (red). Interviewers respect candidates who can articulate this boundary precisely.

Status	Feature	Notes
IMPLEMENTED	Pareto dominance analysis	O(N ²) dominance check, efficiency ranking, COST_EPSILON
IMPLEMENTED	Three-band complexity routing	complexity_low/high thresholds, quality_floor enforced
IMPLEMENTED	PASS/WARN/FAIL verdict logic	Dominated->FAIL, quality_delta/savings->WARN, else PASS
IMPLEMENTED	ROUGE-1 F1 evaluation	Counter intersection, precision+recall, F1 formula
IMPLEMENTED	TF-IDF cosine similarity	Semantic component for quality scoring (BERTScore proxy)
IMPLEMENTED	Append-only JSONL audit log	AuditEvent Pydantic model, open(a) write, one line/event
IMPLEMENTED	Per-task EVAL_WEIGHTS	Summarization: rouge 0.6/semantic 0.4; classification, extraction
IMPLEMENTED	Four model tier configs	large/medium/small/local with cost+quality baselines
IMPLEMENTED	Complexity scorer	0.50 token + 0.30 vocab_entropy + 0.20 task_baseline
IMPLEMENTED	Macro F1 for classification	Per-class precision/recall, unweighted average
PARTIAL	BERTScore semantic evaluation	TF-IDF proxy implemented; true BERT embeddings not loaded
PARTIAL	Cold start handling	Config.py baselines as priors; no auto-calibration protocol
PARTIAL	Pareto cache invalidation	No versioning; frontier recomputed on each pipeline.run()
ROADMAP	Circuit breaker per model tier	No fallback when tier endpoint fails
ROADMAP	A/B testing framework	No holdout experiment infrastructure
ROADMAP	Complexity auto-calibration	Weights (0.50/0.30/0.20) hardcoded; no workload-specific tuning
ROADMAP	Prompt injection detection	No adversarial input detection before complexity scoring
ROADMAP	Streaming/async routing	Synchronous only; no async model calls
ROADMAP	Multi-modal complexity	Image/audio inputs not scored; text-only
ROADMAP	Provider health monitoring	No per-tier latency/availability tracking

Section 17 — Key Numbers Reference Card

These are the numbers an interviewer will probe. Know every value, what it controls, and what happens when you change it up or down.

Number	What It Controls	What Changes If You Adjust It
0.35	complexity_low threshold	Below this -> local tier. Raise to route more to local; lower to route less.
0.70	complexity_high threshold	Above this -> large tier. Raise to route more to medium; lower to route more to large.
0.75	quality_floor	Minimum acceptable quality. Raise to be more conservative; lower to save more cost.
0.20	cost_savings_min	Minimum savings for PASS. Below 20% savings -> WARN. Raise for stricter cost discipline.
0.10	quality_delta WARN threshold	If realized quality > 10% below large baseline -> WARN. Lower to be more sensitive.
0.50	token_score weight in complexity	Largest factor. Increase if token count is most predictive for your workload.
0.30	vocab_entropy weight in complexity	Domain specificity signal. Increase for domain-specialized workloads.
0.20	task_baseline weight in complexity	Categorical floor by task type. Rarely the dominant factor.
2048	token normalization denominator	$\text{token_score} = \min(n_tokens/2048, 1.0)$. Adjust if your prompts are typically longer.
1e-6	COST_NORMALIZATION_EPSILON	Prevents division by zero in cost normalization. Never change this.
0.001	Routing efficiency denominator guard	$\text{quality}/(\text{norm_cost} * 1000 + 0.001)$. Prevents explosion for zero-cost local tier.
0.91	Large tier quality — summarization	Expected ROUGE-1/semantic weighted score for gpt-4o on summarization.
0.88	Medium tier quality — summarization	Expected score for gpt-4o-mini. Delta from large = 0.03 (3.3% drop).
0.79	Small tier quality — summarization	Expected score for gpt-3.5-turbo. Delta from large = 0.12 (13.2% drop).
0.74	Local tier quality — summarization	Expected score for ollama/mistral. Delta from large = 0.17 (18.7% drop).
0.6/0.4	EVAL_WEIGHTS summarization	ROUGE-1 weight 0.6, semantic weight 0.4. ROUGE dominates.
0.7/0.3	EVAL_WEIGHTS classification	Macro F1 weight 0.7, semantic weight 0.3. F1 dominates.
0.5/0.5	EVAL_WEIGHTS extraction	Exact match weight 0.5, semantic weight 0.5. Equal split.

Appendix A — Annotated Source Code Reference

This appendix walks through the most interview-critical code in InferenceLens. Tracing from concept to code line is the difference between a candidate who built the project and one who skimmed the README.

A.1 `ParetoAnalyzer.analyze()` — The Core Logic (pipeline/pareto.py)

```
def analyze(self, run: ProfileRun) -> ParetoResult:
    # Step 1: Aggregate per-tier stats from ProfileRun
    tier_stats = {} # tier → (avg_cost, avg_quality, avg_latency)
    for tier in run.tiers_profiled:
        tier_stats[tier] = (run.avg_cost(tier), run.avg_quality(tier), run.avg_latency(tier))

    # Step 2: Dominance check – O(M^2) nested loop over M configs
    for tier, (cost, quality, latency) in tier_stats.items():
        status = ParetoStatus.FRONTIER # assume frontier until dominated
        for other_tier, (other_cost, other_quality, _) in tier_stats.items():
            if other_tier == tier: continue
            # Strict dominance: other is cheaper AND at least as good
            if other_cost < cost and other_quality >= quality:
                status = ParetoStatus.DOMINATED
                dominated_by = other_tier
                break
            # Cost-tied dominance: same cost but strictly better quality
            if abs(other_cost - cost) < 1e-9 and other_quality > quality:
                status = ParetoStatus.DOMINATED
                dominated_by = other_tier
                break

    # Step 3: Compute routing efficiency for each config
    normalized_cost = max(cost, self.COST_NORMALIZATION_EPSILON) # 1e-6
    routing_efficiency = quality / (normalized_cost * 1000 + 0.001)
    # Epsilon 0.001 prevents /0 for local ($0). Scale 1000 normalizes cost
    # to same order as quality (0-1). Higher = more quality per dollar.
```

Key interview lines: the nested dominance check (`other_cost < cost AND other_quality >= quality`), the tie-breaking case (`abs(cost_diff) < 1e-9`), and the `routing_efficiency` formula. For $M=4$ configs, this is 12 comparisons total — negligible.

A.2 `RoutingRecommender._select_tier_for_band()` (pipeline/router.py)

```
def _select_tier_for_band(self, band, frontier_points, large_point):
    quality_floor = self.thresholds.quality_floor # 0.75 from config.py

    if band == ComplexityBand.LOW:
        # Cheapest frontier config above quality floor
        eligible = [p for p in frontier_points if p.avg_quality >= quality_floor]
        if not eligible: eligible = frontier_points # floor not achievable: use all
        eligible.sort(key=lambda p: p.avg_cost_usd) # cheapest first
        return eligible[0].tier, eligible[0]

    elif band == ComplexityBand.MEDIUM:
        # Middle quality point on frontier
        if len(frontier_points) >= 3:
            point = frontier_points[len(frontier_points) // 2]
```

```

elif len(frontier_points) >= 2:
    point = frontier_points[1] # second-best
else:
    point = frontier_points[0] # only option
    return point.tier, point

else: # ComplexityBand.HIGH
    # Best quality on frontier (frontier sorted by quality DESC)
    return frontier_points[0].tier, frontier_points[0]

```

Key insight: `frontier_points` is sorted by quality DESCENDING. So `frontier_points[0]` is always the highest-quality frontier point. For LOW band, we sort by cost ASCENDING among eligible points, picking cheapest. For HIGH band, quality wins without cost constraint.

A.3 `QualityEvaluator._compute_metrics()` (pipeline/evaluator.py)

```

def _compute_metrics(self, prompt_id, task_type, tier, predicted, ground_truth):
    sem_sim = _vectorizer.cosine_similarity(predicted, ground_truth)
    weights = EVAL_WEIGHTS.get(task_type, {'semantic': 1.0}) # fallback

    if task_type == 'summarization':
        rouge = _rouge1_f1(predicted, ground_truth)
        primary = weights['rouge'] * rouge + weights['semantic'] * sem_sim
        # weights: rouge=0.6, semantic=0.4 (from config.py EVAL_WEIGHTS)

    elif task_type == 'classification':
        f1 = _macro_f1(predicted, ground_truth) # binary label match
        primary = weights['f1'] * f1 + weights['semantic'] * sem_sim
        # weights: f1=0.7, semantic=0.3

    elif task_type == 'extraction':
        em = _extraction_score(predicted, ground_truth) # field-level
        primary = weights['exact_match'] * em + weights['semantic'] * sem_sim
        # weights: exact_match=0.5, semantic=0.5

    else:
        # Unknown task type → length-based proxy fallback
        primary = _length_proxy(predicted, ground_truth)
        fallback_used = True

```

The `EVAL_WEIGHTS` dictionary in `config.py` controls these weights. They are configurable without changing `evaluator.py`. Key: classification weight on `f1` is 0.7 (highest) because label correctness is non-negotiable. Summarization semantic weight is 0.4 (highest relative to primary) because paraphrase is acceptable in summaries.

A.4 `AuditLogger` — append-only write pattern (pipeline/audit.py)

```

class AuditLogger:
    def __init__(self, log_path=None):
        self.log_path = Path(log_path or AUDIT_LOG_PATH)
        self._buffer: List[AuditEvent] = []
        self.log_path.parent.mkdir(parents=True, exist_ok=True) # ensure dir

    def log(self, event: AuditEvent) -> None:
        self._buffer.append(event)
        # Write immediately – append-only, no seek
        with open(self.log_path, 'a', encoding='utf-8') as f:
            f.write(event.to_jsonl() + '\n') # atomic line write

```

```
def flush(self) -> None:
    # Buffer is already written; this clears in-memory buffer only
    self._buffer.clear()
```

Key: `open(path, 'a')` opens in append mode — each write adds a line without seeking or overwriting. 'a' mode is atomic at the OS level for single-writer scenarios. Each event is serialized via `AuditEvent.to_jsonl()` which calls Pydantic's `model_dump_json()`. The in-memory `_buffer` enables query methods (`filter_by_task`, `get_verdicts`) without re-reading the file.

A.5 ROUGE-1 F1 Implementation (pipeline/evaluator.py)

```
def _rouge1_f1(hypothesis: str, reference: str) -> float:
    hyp_tokens = re.findall(r'\b\w+\b', hypothesis.lower())
    ref_tokens = re.findall(r'\b\w+\b', reference.lower())
    if not ref_tokens: return 0.0 # empty reference → undefined
    hyp_counter = Counter(hyp_tokens)
    ref_counter = Counter(ref_tokens)
    # Counter & = element-wise minimum (overlap)
    overlap = sum((hyp_counter & ref_counter).values())
    precision = overlap / len(hyp_tokens) if hyp_tokens else 0.0
    recall = overlap / len(ref_tokens)
    if precision + recall == 0: return 0.0
    return round(2 * precision * recall / (precision + recall), 4) # F1
```

Interview note on Counter &: Python's Counter & operator computes element-wise minimum. `Counter({'the':3,'cat':2}) & Counter({'the':2,'cat':4,'dog':1}) = Counter({'the':2,'cat':2})`. This correctly handles repeated words: if 'the' appears 3 times in hypothesis and 2 times in reference, the overlap credit is $\min(3,2)=2$.

Appendix B — Mock Interview Simulations

Three interview scenarios at increasing difficulty. Practice these until the reasoning flows naturally — the goal is to internalize WHY, not memorize words.

B.1 Phone Screen (30 minutes) — L4 ML Engineer Role

Interviewer: *Tell me about a project where you optimized inference costs.*

Candidate: I built InferenceLens, an inference cost and quality tradeoff auditor for AI systems. The core problem: most teams route all API calls to the largest model by default. When gpt-4o handles a sentiment classification that gpt-3.5-turbo would solve at 94% quality for 97% less cost, that's a routing failure with no paper trail. InferenceLens finds it by profiling each model tier against real tasks, computing the Pareto frontier across cost and quality dimensions, and generating IF/THEN routing rules backed by empirical evidence. Every routing decision is logged in an append-only JSONL audit trail so you can always explain why a model was chosen.

Interviewer: *How do you measure quality if you can't make live API calls?*

Candidate: The current implementation uses a synthetic dataset with pre-computed quality scores, calibrated from published model benchmarks. For summarization, quality is ROUGE-1 F1 — unigram overlap between the model output and a reference summary. For classification, it's macro F1, which is binary label correctness for single-label tasks. For extraction, it's field-level exact match with partial credit. The framework is designed so that live API integration is a drop-in replacement — the same QualityEvaluator class receives actual model outputs instead of pre-computed scores. I'm upfront that the current implementation uses simulated quality — the value is in the routing framework and Pareto analysis, not the specific quality numbers.

Interviewer: *What's the most important limitation of your complexity scorer?*

Candidate: It's a proxy, not a ground-truth predictor of task difficulty. It uses three features: normalized prompt token count, vocabulary entropy (type-token ratio), and a fixed task-type baseline. The biggest limitation is that it's computed from prompt features alone — it can't detect domain-specific difficulty that isn't captured by vocabulary diversity. A 200-token prompt about quantum error correction might score 'MEDIUM' complexity when it actually requires a large model for correct domain reasoning. The correct mitigation is to validate the complexity score against actual quality outcomes via A/B testing — route some calls with InferenceLens rules, compare quality to the baseline, and recalibrate the scorer if there are systematic mismatches.

B.2 Technical Deep-Dive (60 minutes) — L5 ML Engineer Role

Interviewer: *Walk me through the Pareto dominance check. Can you trace the code?*

Candidate: Sure. In ParetoAnalyzer.analyze(), after aggregating per-tier stats, I run a nested loop over all (tier, other_tier) pairs. For each pair, I check: is other_tier cheaper than tier AND at least as good quality? If yes, tier is DOMINATED. There's also a tie-breaking case: if costs are within 1e-9 (floating-point equal) but quality is strictly better, also DOMINATED. For M=4 tiers, this is 12 comparisons. Applied to summarization: medium (\$0.00021, 0.88) vs small (\$0.00050, 0.79). Is medium cheaper than small? Yes (\$0.00021 < \$0.00050). Is medium quality >= small? Yes (0.88 >= 0.79). So small is DOMINATED. No other pair dominates medium — large has better quality but higher cost; local has lower cost but lower quality. Medium is FRONTIER.

Interviewer: *Why is gpt-3.5-turbo (small) dominated by gpt-4o-mini (medium) when gpt-3.5 is newer in some benchmarks?*

Candidate: InferenceLens uses calibrated quality baselines from config.py, not live API measurements. The baselines reflect relative performance on the three task types (summarization, classification, extraction) as observed in published model benchmarks at the time of calibration. The specific numbers are: small achieves 0.79/0.86/0.77 for sum/cls/ext vs medium at 0.88/0.91/0.85. On these three task types, medium is strictly better. If

you have domain-specific evidence that gpt-3.5-turbo outperforms gpt-4o-mini on your specific tasks, you'd update the QUALITY_BASELINES in config.py and re-run the Pareto analysis. The framework accommodates this — it's a configuration change, not a code change. This is why I always caveat: 'these are simulated baselines; live profiling on your domain is the production-grade approach.'

Interviewer: Your routing efficiency formula divides quality by cost. Doesn't this make the local model infinitely efficient?

Candidate: Great question — that's exactly why the epsilon (0.001) is there. The formula is $\text{quality} / (\text{cost} * 1000 + 0.001)$. For local model (cost=\$0): $\text{efficiency} = 0.74 / (0 * 1000 + 0.001) = 0.74 / 0.001 = 740$. For medium (\$0.00021): $0.88 / (0.00021 * 1000 + 0.001) = 0.88 / 0.211 = 4.17$. So local has ~177x higher routing efficiency than medium. This is intentional — the formula correctly identifies that free inference is extremely cost-efficient. But routing efficiency is only one input to the routing decision. The quality floor (0.75) is the gate: local quality of 0.74 is BELOW the floor for summarization, so it cannot be recommended despite having the highest efficiency score. The floor constraint prevents the efficiency formula from over-recommending low-quality free models for tasks where quality matters.

B.3 System Design (45 minutes) — Staff Engineer Role

Interviewer: Design InferenceLens as a live production routing service for 10M daily calls.

Candidate: I'd split it into four layers. First, the profiling layer: a background service that samples 0.1% of production traffic (10K calls/day), routes those calls to all 4 model tiers, measures actual quality against a reference answer, and stores profiling results in a profiling_results table partitioned by (task_type, date). This gives continuously updated quality baselines from real production data. Second, the analysis layer: a daily Pareto analysis job reads the 7-day rolling profiling window, recomputes the frontier, regenerates routing rules, and writes them to a routing_rules table with effective_from timestamp. Third, the routing layer: a FastAPI service reads routing rules from Redis (TTL=1h), computes complexity_score for each incoming prompt in <1ms, returns the routing recommendation (tier, rule_id, confidence). Zero model calls in this path. Fourth, the audit layer: every routing decision appended to a Kafka topic, consumed by a warehouse loader, queryable in BigQuery within 5 minutes. Alert: if WARN verdict rate exceeds 5% for any task type, trigger immediate re-analysis.

Interviewer: How do you ensure routing rules don't degrade quality silently?

Candidate: Three mechanisms. First, continuous quality sampling: the 0.1% profiling sample gives daily quality measurements. If the rolling 7-day average quality for any tier drops below the floor, trigger re-profiling with an expanded sample (1% for 3 days) to confirm the regression. Second, shadow mode: for the first 30 days after any rule change, run shadow routing — apply the new rules but log the routing decision without changing actual model behavior. Compare the shadow quality distribution to the baseline. Only activate the new rules if shadow results confirm quality floor is maintained. Third, canary deployment: activate new rules for 5% of traffic first. Monitor quality metrics for 24 hours before full rollout. If quality drops below threshold, automatic rollback to previous rules via Redis rule version switch.

Appendix C — Comparison Matrix vs Alternatives

Be prepared to compare InferenceLens to any of these alternatives.

Alternative	Strengths	Why InferenceLens is Different
OpenRouter / LiteLLM	API-level routing, load balancing, failover — production-ready	No empirical quality measurement, no Pareto analysis, no audit trail with routing rationale. InferenceLens generates the routing rules that OpenRouter enforces. Complementary tools.
Model cascading	Routes to small model first, escalates on low confidence	Requires an initial model call to assess confidence (latency + cost overhead). InferenceLens routing is pre-call and zero-cost. Cascading better when confidence can be estimated from initial output; InferenceLens better when prompt features predict difficulty.
Manual routing heuristics	Simple, easy to implement, no overhead	No empirical validation, no audit trail, degrades silently as models change. InferenceLens replaces hand-tuned rules with evidence-backed rules that can be justified.
PromptLayer / Helicone	Inference observability, cost tracking, prompt versioning	Observability tools, not routing decision tools. They log what happened; InferenceLens recommends what should happen. Complementary: InferenceLens consumes observability data to calibrate routing rules.
Always-large routing	Maximum quality guarantee, operationally simple	Correct for high-stakes tasks. Unnecessarily expensive for the 70-80% of tasks where smaller models achieve equivalent quality. InferenceLens identifies that 70-80% precisely.
Semantic routing (embedding-based)	Routes based on query-topic similarity to known task types	Requires embedding model for every call. Cannot route on cost/quality tradeoff — routes on topic match only. No empirical quality measurement. No Pareto analysis.

C.2 When NOT to use InferenceLens

- **When all tasks require maximum quality:** High-stakes medical, legal, or financial generation where quality degradation is unacceptable. Use always-large routing with InferenceLens as a monitoring layer only.
- **When you have no prompt structure:** If `task_type` cannot be classified (free-form multi-turn conversations with mixed intents), the complexity score and routing rules lose their precision. Task-type routing requires identifiable task boundaries.
- **When latency is the dominant constraint:** InferenceLens optimizes cost/quality tradeoff. If latency SLA is below the local model's 890ms average, the Pareto analysis needs a latency dimension — not currently supported.
- **When production data is unavailable for profiling:** Synthetic baselines from `config.py` are proxies. Domain-specific tasks (legal contract extraction, medical diagnosis) require live profiling on domain-representative prompts.

Appendix D — Quick-Reference Card

D.1 The 5 Things to Say in the First 90 Seconds

- InferenceLens makes implicit AI inference cost/quality tradeoffs **explicit, measurable, and defensible**.
- It profiles model tiers against real tasks, computes the **Pareto frontier** (no dominated config is ever recommended), and generates **IF/THEN routing rules** per complexity band.
- Every routing decision is logged in an **append-only JSONL audit trail** — you can always explain why a model was chosen.
- Complexity score is computed **before any model call** — zero overhead, deterministic, loggable.
- Current implementation uses **simulated inference** (seed=42). Live API integration is the designed next step.

D.2 Numbers to Know Cold

Item	Value and Context
Quality floor	0.75 — Minimum acceptable quality for any routing recommendation
Savings floor	20% — Minimum cost savings required to route down a tier
Complexity bands	0.35 / 0.70 — Simple <0.35, Standard 0.35-0.70, Complex >=0.70
Complexity weights	0.50 / 0.30 / 0.20 — Token count / vocab entropy / task baseline
Task baselines	0.15 / 0.10 / 0.05 — Extraction / Classification / Summarization
Dataset	300 prompts, seed=42 — 100 per task type, 4 tiers, deterministic
Tests	48 passing — Pipeline integration, Pareto, routing, evaluator
WARN threshold	>10% quality delta OR <20% savings — <code>_compute_verdict()</code> in <code>router.py</code>
FAIL trigger	Dominated config in use — FAIL = immediate intervention required
Audit event ID	<code>evt_{10-char hex}</code> — UUID-based, unique per inference call

D.3 Pareto Dominance (for whiteboard)

```
Config A dominates Config B iff:
A.cost < B.cost AND A.quality >= B.quality
OR (same cost) AND A.quality > B.quality

Config is FRONTIER iff no other config dominates it.
Config is DOMINATED iff at least one config dominates it.

routing_efficiency = quality / (cost * 1000 + 0.001)
# 0.001 prevents /0 for local ($0 cost)
# * 1000 scales cost to same order as quality

Tier selection:
LOW band: cheapest frontier config above quality_floor=0.75
MED band: middle quality frontier point
HIGH band: highest quality frontier point
```

D.4 ROUGE-1 F1 (for whiteboard)

```
tokenize(text) = re.findall(r'\b\w+\b', text.lower())
```

```
overlap = sum((Counter(hyp_tokens) & Counter(ref_tokens)).values())
precision = overlap / len(hyp_tokens)
recall = overlap / len(ref_tokens)
F1 = 2 * P * R / (P + R)
```

Example: hyp='the cat sat on mat', ref='the cat is on the mat'

overlap={the:1, cat:1, on:1, mat:1} = 4

precision=4/5=0.80, recall=4/6=0.667, F1=0.727

D.5 The One Sentence That Frames the Project

"InferenceLens makes the cost/quality Pareto frontier for AI inference explicit, measurable, and defensible — profiling model configurations against real tasks, identifying dominated configs that should never be used, and generating routing rules backed by empirical evidence with a complete audit trail."

Every answer should ladder up to this statement. When in doubt, return to the thesis question: 'How does an AI system know it's working correctly when cost and quality are competing constraints?' That's the question InferenceLens answers.